



Lab Assignment – 2.1 (SQL Data Definition Language – DDL)

Department: CSE

Subject: DIS Lab
Code: 22CPP217

Run the following command before starting database to create user for yourself -

```
SQL
> sudo mysql

> CREATE USER '<your_username>'@'localhost' IDENTIFIED BY '<your_passowrd>';

> GRANT ALL PRIVILEGES ON *.* TO '<your_username>'@'localhost';

> exit;
```

Now start mysql again with your credentials > **sudo mysql -u <your_username> -p**

For the people facing problem in logging, do a clean installation by executing following commands in terminal -

```
Shell

# Uninstall packages:
sudo systemctl stop mysql
sudo apt purge mysql-server mysql-client mysql-common
sudo apt purge mysql-server-core-* mysql-client-core-*
sudo apt autoremove
sudo apt autoclean

# delete data and config
sudo rm -rf /etc/mysql /var/lib/mysql /var/log/mysql

# reinstall
sudo apt update
sudo apt install mysql-server
```

Now run the command in first code block, and create user.

Objective: The objective of this experiment is to learn **Data Definition Language** (DDL) by designing robust database schemas that utilize appropriate data types and integrity constraints. Also learn how to modify (**ALTER**) an active database structure to accommodate changing requirements and debug constraint violations.

[Reference: MySQL 8.x Official Documentation – SQL Statements \(DDL\)](#)

[Use the table provided in this link](#) for following questions:

Question 1: Design and Implementation of a Robust Schema You are tasked with digitizing university student records. The legacy data contains formatting anomalies (e.g., IDs with leading zeros) and potential errors. Create a table named **Student_Registry** that enforces data integrity using strict architectural requirements.

- Data Type Selection: Choose the correct data type for the ID column to ensure values are preserved.
- Integrity Constraints: Apply the following constraints during table creation:
 - Dept_Name: If a user does not provide a department name, the database must automatically assign 'General'.
 - Tot_Cred: A student's total credits can be 0 but must never be negative. Apply a **CHECK** constraint to enforce this.
- Data Population: Insert the provided dataset to verify your design. Ensure that the IDs retain their zeros and the empty entry defaults to the '**General**' department.

Question 2: Schema Evolution and Integrity Enforcement After the system goes live, the university updates its rules. You must modify the active database structure using **ALTER** commands without losing existing data.

- Constraint Conflict: The university decides to cap undergraduate credits at 100.
 - Write and execute an **ALTER TABLE** command to add a **CHECK** constraint ensuring **tot_cred <= 100**.
 - Capture a screenshot of the error message.
 - Write an **UPDATE** query to correct the invalid data (set credits > 100 to exactly 100), then re-run the **ALTER** command successfully.
- Schema Expansion: Add a new column email to the table. Apply a constraint to ensure that no two students can share the same email address.
- Structural Refactoring: Rename the column <tot_cred> to <total_credits> and increase the size of the Name column to 40 characters to accommodate longer names.
- Structural Cleanup: Perform the following two operations and record the difference in their behavior:
 - Execute **TRUNCATE TABLE** and check if the table structure exists.
 - Execute **DROP TABLE** and check if the table structure exists.

SQL

SQL Commands Syntax:

```
// creating, show and selecting database
CREATE DATABASE database_name;
SHOW DATABASES;
USE database_name;

// table info
SHOW TABLES;
SHOW TABLES FROM database_name;
DESCRIBE table_name;

// create table
CREATE TABLE table_name ( column_name datatype [constraint], ... );
PRIMARY KEY (column_name);
DEFAULT value;
CHECK (condition);

//inserting data
INSERT INTO table_name (column_list) VALUES (...);

// adding column
ALTER TABLE table_name ADD column_name datatype [constraint];

// adding constraints
ALTER TABLE table_name ADD CHECK (condition);
ALTER TABLE table_name ADD CONSTRAINT constraint_name CHECK (condition);
ALTER TABLE table_name ADD UNIQUE (column_name);

// rename/modify column
ALTER TABLE table_name RENAME COLUMN old_name TO new_name;
ALTER TABLE table_name MODIFY column_name datatype;

// update data
UPDATE table_name SET column_name = value WHERE (condition);

// delete data/table
TRUNCATE TABLE table_name;
DROP TABLE table_name;

// display table
SELECT * FROM table_name;
SELECT column_name FROM table_name WHERE (condition);
```

Some Common Constraints:

- **PRIMARY KEY:** Uniquely identifies each record.
- **NOT NULL:** Ensures a column cannot have a NULL value.
- **DEFAULT 'Value':** Sets a default value if none is provided.
- **CHECK (column_name condition):** e.g., CHECK (age >= 18).